Spark vs Ray: The Data Engineering Duel

DA3408 AI Operations Lab, Assignment 3 | DA24B025 | 05 October 2026

1. Objective and dataset

The same four-step preprocessing pipeline (ingestion, cleansing, transformation, export) was implemented in Apache Spark (`spark_clean.py`) and Ray Data (`ray_clean.py`), deployed on two-worker clusters and benchmarked on the NYC TLC Yellow Taxi trip records for January to April 2026, joined with the TLC taxi zone lookup table.

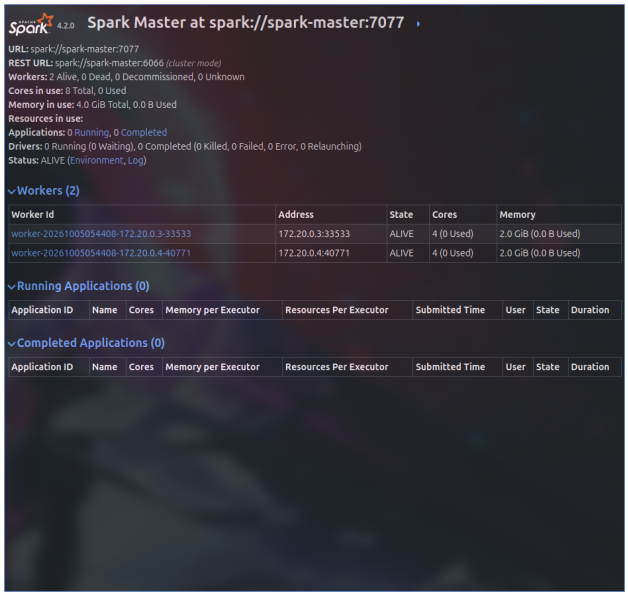
Files	Rows	Columns	Parquet on disk	Uncompressed in memory (Arrow)
4	14,908,446	20	0.26 GB	2.25 GB

Parquet is columnar and compressed, so the roughly 2 GB working set the assignment asks for corresponds to about a quarter of a gigabyte on disk. The raw data is genuinely unclean: about 26% of rows have nulls in passenger count, rate code and fee columns, and there are trips that end before they start, zero or negative distances and fares, and pickups dated as early as 2001.

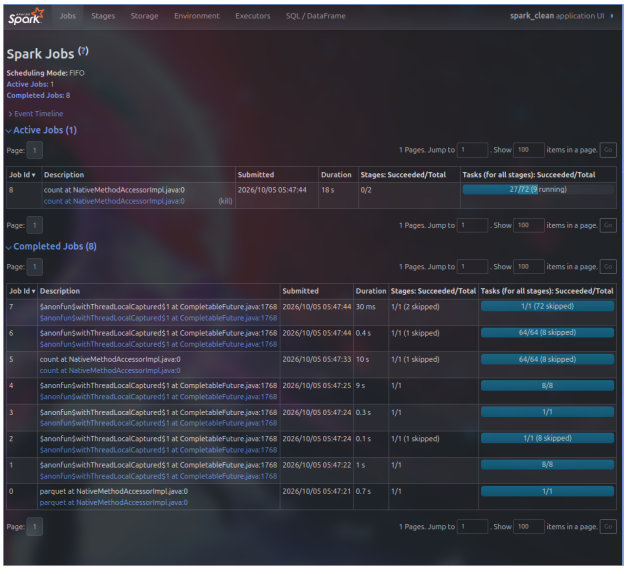
2. Cluster orchestration

Both clusters run from one Docker image (Python 3.12, OpenJDK 17, pyspark 4.2.0, ray 2.58.0), so the runtime is identical. Each framework gets its own Docker Compose file and private bridge network; containers resolve each other by service name. Clusters were run one at a time so that neither competed with the other for the 14 GB laptop. Data, output and scratch (spill) directories are bind-mounted at the same paths on every node.

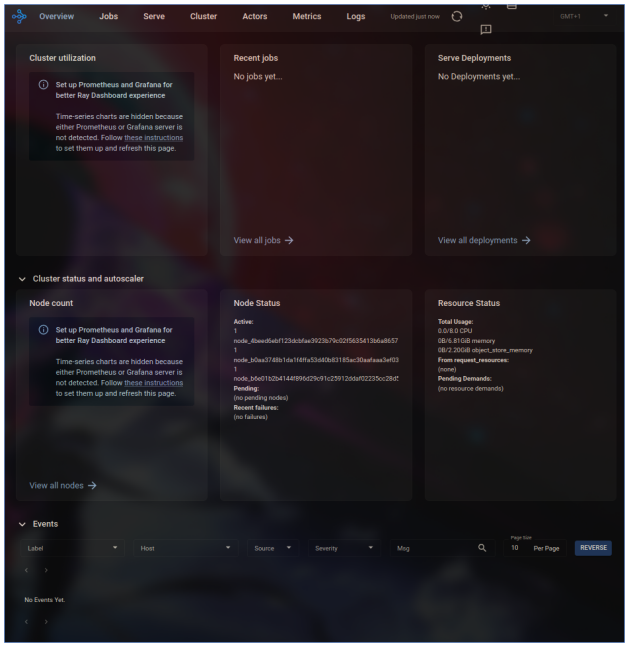
Role	Spark	Ray	CPU / memory limit
Coordinator	spark-master (Master, port 7077, UI 8080, driver UI 4040)	ray-head (--num-cpus=0, GCS 6379, dashboard 8265)	2 CPU / 2 GB
Worker 1	spark-worker-1 (--cores 4 --memory 2g)	ray-worker-1 (--num-cpus=4, 1 GB object store)	4 CPU / 4 GB
Worker 2	spark-worker-2 (--cores 4 --memory 2g)	ray-worker-2 (--num-cpus=4, 1 GB object store)	4 CPU / 4 GB
Driver	spark-submit inside spark-master, 1 executor per worker	python inside ray-head, ray.init(address='auto')	-
Spill to disk	SPARK_LOCAL_DIRS=/scratch/<node>	--object-spilling-directory=/scratch/<node>	host /home



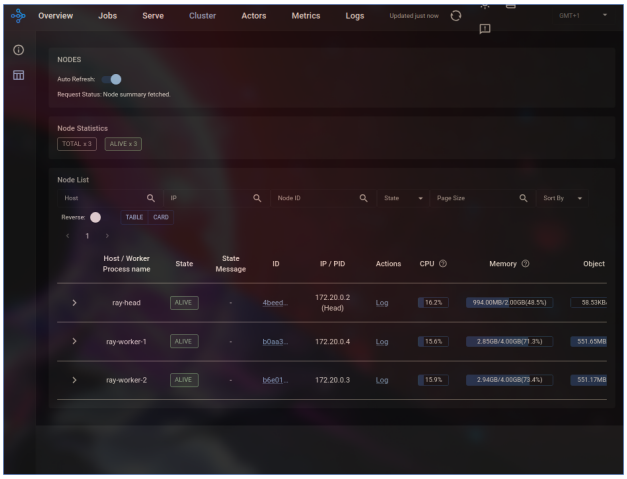
Spark Master UI (port 8080): two ALIVE workers, 4 cores each



Spark application UI (port 4040) while spark_clean.py runs



Ray Dashboard (port 8265): head node and two workers



Ray Dashboard while ray_clean.py runs

3. Pipeline parity

Step	Logic (identical in both)	Spark	Ray Data
Ingestion	Read the four yellow-taxi parquet files	spark.read.parquet	read_parquet
Cleansing	Drop rows with any null; snake_case columns; pickup in Jan-Apr 2026; dropoff after pickup; 0 < distance <= 100; fare, total and passengers > 0; remove exact duplicate rows	na.drop, filter, dropDuplicates()	map_batches; dedup by row hash, repartition(keys), drop_duplicates per block
Heavy join	Pickup and dropoff zone IDs joined to borough / zone / service zone; derive pickup date and hour	broadcast hash join	broadcast join in map_batches
Python UDF	trip_features(): trip duration (min) and average speed (mph)	udf(useArrow=False) or pandas_udf	same function per row, or vectorised
Features	Drop trips > 6 h or > 80 mph; average speed per pickup hour joined back	groupBy + join	groupby().mean() + join
Export	31-column ML-ready parquet	write.parquet	write_parquet

`compare.py` verifies parity without relying on row order: it compares the schemas, row counts, per-column null counts and sums, and an order-independent fingerprint (the wrapping sum of a 64-bit hash of every row, with floats rounded to 6 decimals).

Check	Spark	Ray	Result
Schema (31 columns, names and types)	31 columns	31 columns	match
Row count	10,585,815	10,585,815	match
Null counts per column			31/31 match
Numeric column sums			21/21 match
Row fingerprint	17882023026524830382	17882023026524830382	match

Verdict: the two outputs contain exactly the same rows.

4. Performance analysis

Each script times its stages by materialising the result before the next stage starts (`persist().count()` in Spark, `materialize()` in Ray), so the UDF cost can be isolated. Total time runs from ingestion to the finished parquet export. Values are the mean ± standard deviation over n runs.

Framework	UDF mode	n	Ingest + clean + dedup + join	Python UDF	Hourly features + export	Total
Spark	python	3	25.2 ± 3.2	36.6 ± 1.7	15.0 ± 0.8	76.8 ± 2.6
Spark	pandas	3	30.4 ± 0.4	20.1 ± 0.9	15.3 ± 0.5	65.8 ± 0.1

Ray	python	3	38.0 ± 5.6	10.0 ± 1.5	14.8 ± 0.3	62.8 ± 5.4
Ray	pandas	3	36.7 ± 7.1	5.3 ± 0.5	15.0 ± 0.9	57.0 ± 8.0

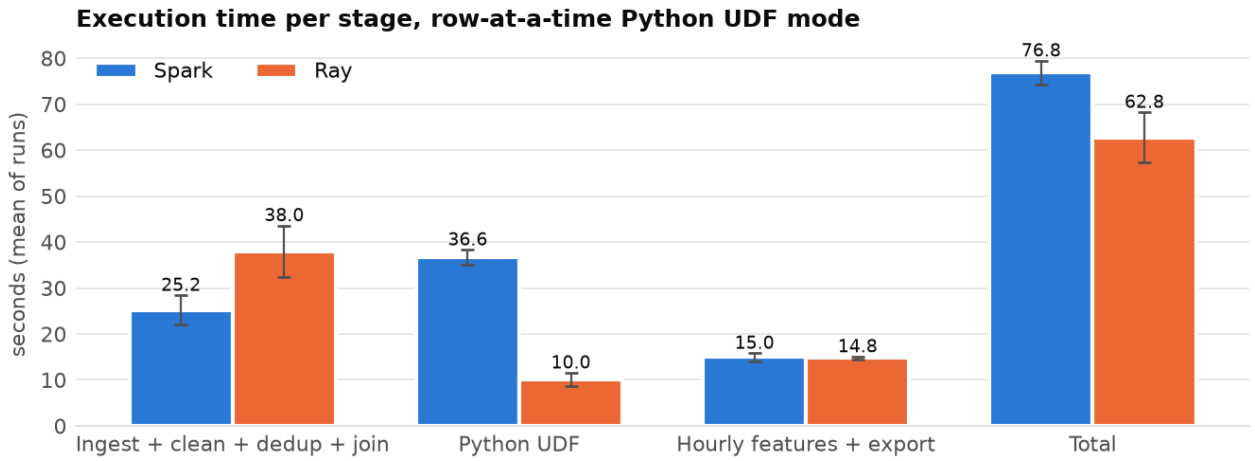


Figure 1. Mean wall-clock time per stage in seconds (error bars: one standard deviation).

Framework	Peak CPU, one worker	Peak memory, one worker	Peak memory, coordinator	Samples
Spark	417% of 400%	3.06 GiB of 4 GiB	0.96 GiB of 2 GiB	280
Ray	429% of 400%	3.69 GiB of 4 GiB	1.50 GiB of 2 GiB	201

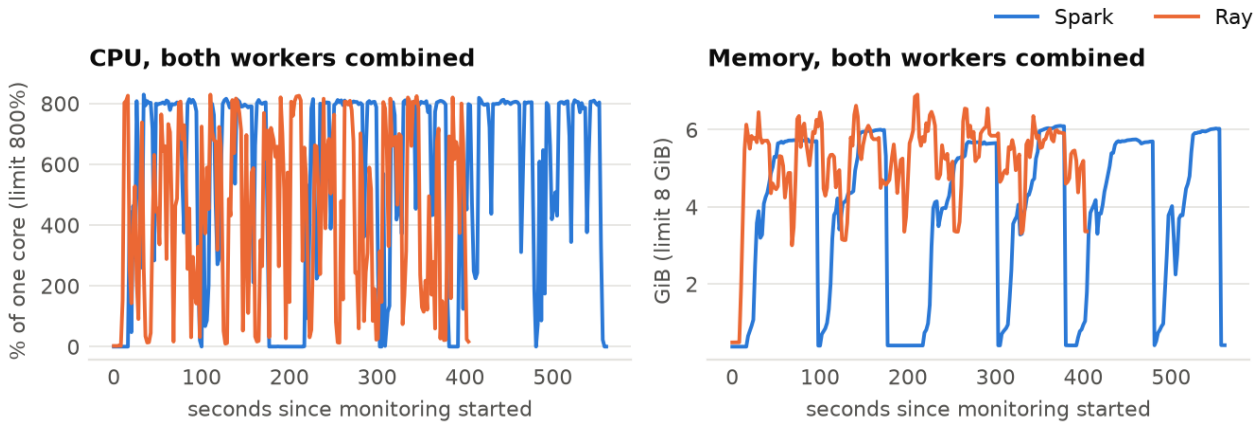


Figure 2. Resource usage sampled with docker stats during the benchmark runs.

Structured work. For ingestion, cleansing, deduplication and the zone join, Spark took 25.2 s against 38.0 s for Ray. Spark executes these steps as native Catalyst/Tungsten operators inside the JVM with whole-stage code generation, and it broadcasts the 265-row zone table automatically. Ray has no global deduplication operator, so the Ray pipeline hashes every row, hash-partitions on that key (an all-to-all shuffle through the object store) and de-duplicates each partition in pandas, which pays Arrow-to-pandas conversion costs on every batch.

5. UDF deep-dive: JVM boundary vs Python-native execution

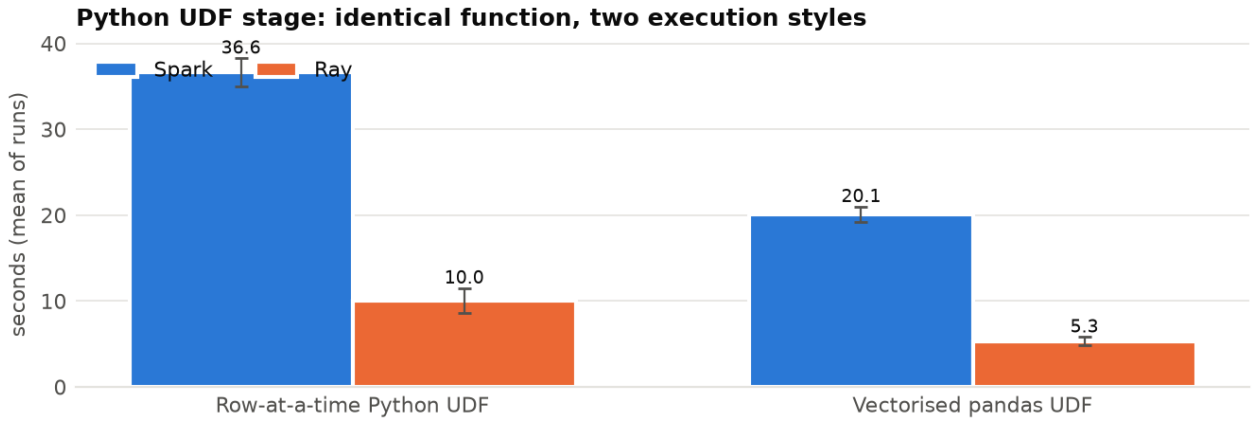


Figure 3. UDF stage time in seconds for the same trip_features() logic.

Both scripts call the same scalar function, `trip_features(pickup, dropoff, distance)`, on the same Python `datetime` and `float` values, so only the execution engine differs.

Row-at-a-time. Spark took 36.6 s and Ray 10.0 s: Ray was 3.7x faster. Spark's classic Python UDF appears in the physical plan as `BatchEvalPython`: each executor JVM serialises rows with Pickle, streams them over a local socket to a separate Python worker process, waits for the results and deserialises them back into the JVM's internal row format. That round trip happens for every row and is pure overhead. Ray's workers are Python processes: a block from the shared-memory object store becomes a pandas batch with near zero-copy, and the function runs in the same process with no language boundary to cross.

Vectorised. With an Arrow-backed `pandas_udf` Spark took 20.1 s; Ray's vectorised `map_batches` took 5.3 s, so Ray was 3.8x faster. Vectorising changed Spark's UDF stage from 36.6 s to 20.1 s and Ray's from 10.0 s to 5.3 s. Arrow replaces Pickle and ships whole column batches instead of single rows, but Spark still copies every batch from the JVM to Python and back, while Ray's data never leaves Python.

Observation. Ray's "Python-native" advantage is real but specific: it removes the cross-runtime serialisation tax that dominates Spark's Python UDFs, so it matters most when the pipeline is mostly custom Python (feature engineering, model inference, NumPy or PyTorch code). It does not make Python faster than Spark's compiled SQL operators, which is why Spark can still win the relational steps of the same pipeline.

6. Winner and recommendation

Winner for this pipeline: Ray, with a mean end-to-end time of 62.8 s against 76.8 s for Spark on identical hardware limits. The stage breakdown matters more than the headline: each framework wins the part of the pipeline it was designed for.

AI-first project: Ray. Ray Data feeds Python code (pandas, NumPy, PyTorch) without a JVM hop, schedules CPU and GPU tasks in the same cluster, and hands data straight to Ray Train and Ray Serve. When most of the compute is custom Python, such as feature engineering, embeddings, batch inference or last-mile preprocessing for training, Ray avoids the serialisation tax measured above.

BI-first project: Spark. SQL-shaped workloads (joins, aggregations, deduplication, reporting tables) benefit from Spark's cost-based optimiser, automatic broadcast joins, mature shuffle and spill handling, and its ecosystem: Spark SQL, JDBC/Thrift for BI tools, Delta Lake, Iceberg and Hive. In testing, Spark spilled gracefully under memory pressure, while Ray's memory monitor killed and retried tasks near the container limit.

7. Performance tuning note (AI attribution)

AI wrote both pipelines and suggested the following optimisations while debugging them on this cluster. Each one was tested before being kept.

- **Ray deduplication.** The first version used `groupby(all 20 columns).count()`. Its hash-aggregate actors used under 1 of the 8 CPUs and the stage ran for more than 10 minutes without finishing. The AI replaced it with a 64-bit row hash, `repartition(64, keys=['row_key'])` and `drop_duplicates` per block, which is the same strategy Spark's dropDuplicates uses. It is still exact, because identical rows always land in the same partition.
- **Ray block size.** `target_max_block_size` was lowered from 128 MB to 32 MB so that pandas conversions inside 4 GB workers stay small. This reduced, but did not eliminate, out-of-memory task retries.

- **Ray object-store budget.** By default Ray Data may use half of the cluster object store. Once the stored stage results filled that budget, the progress log showed `Tasks: 1 [backpressured]` and the UDF stage ran on 1 of the 8 CPUs. Setting `override_object_store_memory_limit_fraction=4.0` restores 8 parallel tasks and lets the excess spill to disk, which mirrors Spark's MEMORY_AND_DISK persist.
- **Worker memory.** Raised from 3 GB to 4 GB for both frameworks after a Spark worker peaked at 3.05 GiB in pandas-UDF mode.
- **Spill location.** The root disk had 7.8 GB free and Ray aborted with OutOfDiskError when it was 95% full. Spark local dirs and Ray's object-spilling directory were moved to a bind-mounted folder on /home.
- **Fair UDF inputs.** Ray's row-at-a-time path first passed pandas Timestamp objects, which are much slower to subtract than datetime objects. It now passes plain Python datetime and float values, exactly what Spark's UDF receives.
- **Spark UDF mode.** `udf(useArrow=False)` is set explicitly so that the row-at-a-time mode measures the classic Pickle-based path. The file list is expanded in Python rather than as a glob, to avoid a misleading FileNotFoundException in the logs.

Code, compose files and full reproduction steps: `https://github.com/samrudh123/DA3408-Assignment-3`. AI usage is declared in AI_DISCLOSURE.md.